

Project 3 — DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely

Title: DocuMind Smart Document Agent Ask Your Documents Anything Safely

Type: Multi-agent (3 agents) · **Stack:** Python, LangGraph, RAG (Chroma/FAISS) · **Level:** Intermediate **Tagline:** A 3-agent LangGraph system — Retriever, Reasoner, and Approval — that grounds every answer in your documents and pauses for a human before acting.

Overview

DocuMind is a 3-agent LangGraph system that answers questions grounded in your own documents — a policy PDF, an internal wiki, a contract — instead of the model's general training knowledge, and pauses for a human to approve before taking any risky follow-on action. It solves a real problem: a model can sound completely confident about your specific documents while being flat wrong about them, and letting an agent take a real action (sending an email, issuing a refund) with no human check at all is genuinely risky. Anyone who needs answers grounded in private documents, with a human sign-off on the actions that matter but not on every single run, would want something like this.

Features

- Retrieval-augmented answering grounded in your own document set, backed by a Chroma/FAISS vector store
- Retriever agent that judges the quality of its own search results and routes to a "can't ground this" path when nothing relevant was found
- Reasoner agent that only answers from what was actually retrieved — never fills gaps from its own memory
- Approval agent that auto-clears safe, well-grounded drafts and only pauses for a human via `interrupt()` when its checks aren't confident
- Full pause-and-resume support via LangGraph checkpointing — state survives a process restart, not just an in-memory pause
- Typed, inspectable graph state at every step, so the whole run can be drawn and explained from the code

Tech Stack

- Python

- LangGraph
- LangChain
- Chroma / FAISS (vector store)
- OpenAI API
- Pydantic

Prerequisites

- Comfortable with RAG basics: vector stores, embeddings, and retrieving top matching chunks
- Familiar with LangGraph fundamentals — state, nodes, edges, and checkpoints
- Comfortable with async Python
- Can build a basic tool-calling agent loop, since this project rebuilds one as a graph

Architecture

A LangGraph `StateGraph` carries one typed state object through three nodes in sequence: **Retriever** (searches the vector store and judges whether what it found is actually good enough to answer from), **Reasoner** (drafts an answer using only what the Retriever found), and **Approval** (runs automatic checks on the draft and calls `interrupt()` to pause for a human only when needed). A conditional edge routes around the Reasoner when the Retriever's search wasn't good enough, and a checkpoint lets the whole run be paused mid-graph and resumed later — even in a fresh process — without losing state.

Setup (do this once, before Step 1)

```
cd project_3_documind
python -m venv .venv
source .venv/bin/activate
pip install openai langchain langchain-openai langgraph chromadb python-dotenv py
pip freeze > requirements.txt
```

New here: `langgraph` (the graph tool), `chromadb` (a local vector store for RAG — swap for `faiss-cpu` if you prefer FAISS). Same `.env` setup as before.

What to put in the vector store: you need real text to search before you can test retrieval. A good option is the Hugging Face dataset [rag-datasets/rag-mini-wikipedia](https://huggingface.co/datasets/rag-mini-wikipedia) — it's small, public domain, and built specifically for practicing RAG pipelines like this one, so it embeds fast and runs locally without a huge download. This is a convenience, not a requirement — your own text files (notes, a PDF, a policy doc) work just as well.

Troubleshooting

Problem	Fix
The graph loops forever, because a branch has no real exit rule	Treat every loop in the graph the same way you'd treat a hard step limit on an agent loop — design and test the exit rule on purpose, don't assume it'll "just work"
The state saved at a pause doesn't include what was found, only the original question	Include the found chunks directly in the state shape, so resuming doesn't need to search again or lose its grounding
Search finds nothing relevant, and the graph answers anyway	Route to a clear "can't ground this answer" path when the search results are below a set quality bar, instead of letting it answer without support

How To Build This — Step by Step

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#) · [Step 5](#)

Step 1 — A 2-Node Graph With No Branching, Just to Prove the Mechanics

Project: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely — Step 1 of 5: A 2-Node Graph With No Branching, Just to Prove the Mechanics

What this step does: proves the basic graph mechanics — state, nodes, one edge — actually work, before any agent behavior or branching gets added. Keeps "do I understand the graph basics" separate from everything after it. **Why this step matters:** every step after this one adds nodes and branches to this exact skeleton — if you don't trust that state actually flows from node to node here, you can't trust it once branching and agents make the graph harder to read. **When you'll hit this for real:** the start of every new LangGraph project you'll ever build — always prove the skeleton compiles and runs before adding real logic to it. **Helpful background:** going from a hidden loop to a clear graph, and how state, nodes, and edges work in LangGraph.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build a minimal 2-node `StateGraph` with a typed state shape and one plain edge — no tools, no branching yet. Define the state as a `TypedDict` (or Pydantic model) with named fields up front, even though it's tiny now — every later step in this project only ever adds fields to this same shape, it's never replaced. 2. By hand, trace what the state looks like after each node runs. Check that your own understanding matches what the code actually does, before adding anything else.

Your files after Step 1:

```

project_3_documind_rag_agent/
├─ state.py           → a minimal TypedDict/Pydantic state
├─ nodes.py           → 2 simple node functions
└─ graph.py           → build_graph() with 2 nodes, 1 edge, no branching

```

Step 2 — Your Existing Agent, Rebuilt as a Graph With Saved State

Project: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely — Step 2 of 5: Your Existing Agent, Rebuilt as a Graph With Saved State

What this step does: proves your existing tool-calling agent can be rebuilt as graph nodes with the exact same behavior, and now gains saved state. **Why this step matters:** this is the migration you'll actually do at a real job far more often than a from-scratch build — taking something that already works and moving it onto infrastructure (here, a graph with a checkpointer) that makes it pausable and inspectable, without changing what it does. **What's new vs. Step 1:** real nodes (your existing tool-calling logic) replace the simple placeholder nodes; a conditional edge and a checkpointer get added. **What stays the same:** the tools themselves and the Worker's decisions — you're changing the *container* (loop → graph), not the agent's behavior. **When you'll hit this for real:** any time you outgrow a hand-rolled agent loop and need it to be pausable, resumable, or inspectable — this exact migration, not a rewrite from scratch. **Helpful background:** loops, checkpointers, and `interrupt()` in LangGraph.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Rebuild your existing Worker agent (tool-calling loop) as graph nodes with a conditional edge — same tools, same behavior, now built as a graph. 2. Connect a checkpointer and prove a pause/resume cycle works on this single-agent graph, before adding any more agents. Test the resume in a fresh process, not just the same one still running — a checkpointer that only appears to work because the Python objects are still sitting in memory isn't actually proving the state was saved.

Your files after Step 2:

```

project_3_documind_rag_agent/
├─ state.py
├─ nodes.py           → single-agent tool-calling nodes (from your existing
├─ graph.py           → conditional edge + checkpointer connected
└─ tools.py           → reused from project_2_researchhand

```

Step 3 — A Retriever Agent That Judges Its Own Search Results

Project: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely — Step 3 of 5: A Retriever Agent That Judges Its Own Search Results

What this step does: adds the first new specialist — a node that doesn't just fetch chunks, but decides whether what it found is actually good enough to answer from. **Why this step matters:** a RAG system that always hands its search results to the answerer, good or bad, is how you get a confident answer built on nothing — judging relevance before answering is what actually earns the word "grounded." **What's new vs. Step 2:** a new Retriever node appears, only run when needed; the state grows to carry found chunks, not just the question. **What stays the same:** Step 2's checkpointer and the core graph-building pattern — you're adding a node to an already-proven base, not rebuilding it. **When you'll hit this for real:** any "answer questions about my documents" feature request — this is the exact node that turns a plain agent into a RAG agent. **Helpful background:** vector stores and searching for the top matches.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build the **Retriever agent** as its own node: wraps `08_rag`'s `retrieve()`, plus a step that judges relevance (is what came back actually useful? if not, route to a "can't ground this" path instead of forcing an answer downstream). Write the relevance bar down as an explicit, checkable rule (a similarity-score cutoff, or a concrete judged question like "do these chunks actually mention the thing being asked?") before you test — a vague "does this look okay?" bar just gets adjusted after the fact to match whatever the first test happened to do. 2. Wire it in as a conditional step in the Step 2 graph — some tasks skip it entirely, same idea as conditional routing in a graph. 3. Test it: a question the knowledge base can answer, and one it can't — confirm the "can't ground this" path actually runs on the second one.

Your files after Step 3:

```
project_3_documind_rag_agent/
├─ state.py           → now includes found chunks, not just the question
├─ graph.py           → conditional routing to the Retriever node
├─ retriever.py       → reused from 08_rag
├─ agents/
│   └─ retriever_agent.py → search + relevance-judging node
└─ nodes.py
```

Step 4 — A Reasoner Agent That Only Answers From What the Retriever Found

Project: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely — Step 4 of 5: A Reasoner Agent That Only Answers From What the Retriever Found

What this step does: adds the specialist that actually writes an answer, and only from grounded material — keeping "did we find good context" (Step 3) separate from "did we use it correctly" (this step). **Why this step matters:** search quality and answer quality fail for different reasons and need different fixes — separating them into two nodes means when an answer is wrong, you can tell in seconds whether the Retriever found the wrong thing or the Reasoner used the right thing badly. **What's new vs. Step 3:** a new Reasoner node uses the Retriever's output; the graph now has

search → answer as a real sequence. **What stays the same:** the Retriever from Step 3 is untouched — Step 4 just adds something that uses its output. **When you'll hit this for real:** the moment a RAG system's answers start drifting from what was actually retrieved — this separation (search quality vs. answer quality) is exactly how you'd diagnose and fix that. **Helpful background:** how agent workflow pieces fit together.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build the **Reasoner agent**: takes the Retriever's output plus the task, and writes a grounded draft answer — clearly told not to answer beyond what was found. Spell out explicitly what it should say when the found chunks don't actually cover the question — an unprompted model tends to quietly fill the gap from what it remembers rather than admit the retrieved material falls short. 2. Wire search → answer as a real edge sequence. 3. Test it: check that the draft actually reflects the found content, not just believable-sounding text — check this by hand against a couple of test cases, not just "it ran without error."

Your files after Step 4:

```
project_3_documind_rag_agent/
├─ state.py
├─ graph.py                → search → answer sequence wired up
├─ retriever.py
├─ agents/
│   ├─ retriever_agent.py
│   └─ reasoner_agent.py    → grounded-answer node
└─ nodes.py
```

Step 5 — An Approval Agent That Only Asks a Human When It Really Needs To (final: 3 agents)

Project: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely — Step 5 of 5: An Approval Agent That Only Asks a Human When It Really Needs To

What this step does: adds the last specialist — a gate that automatically clears low-risk drafts, and only pauses for a real human when its own checks aren't enough — completing the full 3-agent team. **Why this step matters:** full automation is too risky for consequential actions, but a human check on every single run is too slow to actually be used — a gate that only escalates when it's genuinely unsure is the difference between a safety feature people keep and one they route around. **What's new vs. Step 4:** a new Approval node appears with an `interrupt()` gate; routing now branches based on the approval outcome. **What stays the same:** the Retriever and Reasoner from Steps 3-4 don't change — Approval only watches the Reasoner's draft, it doesn't rewrite it. **When you'll hit this for real:** any agent about to take a consequential, hard-to-undo action — sending something, spending money, deleting data — where full automation is too risky but a human-in-the-

loop for every run is too slow. **Helpful background:** using `interrupt()` to pause for a human, and the Sequential and Supervisor multi-agent patterns.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build the **Approval agent**: runs automatic checks on the Reasoner's draft (like — does it actually cite found content, does it avoid a "risky action" without support) — only calls `interrupt()` to ask a real human when its own checks aren't clear, or the action is genuinely important; otherwise it approves on its own and the graph continues. Write the auto-approval checks as an explicit checklist the code evaluates one item at a time, not one vague "is this okay?" model call — a checklist is what lets you point to exactly which check failed when a draft gets wrongly escalated, or wrongly waved through. 2. Wire the final routing: task → Retriever (if needed) → Reasoner → Approval → (approved on its own: done) or (interrupt: pause for a human) → resume → final answer. 3. Test the full cycle, plus the "search found nothing" path (Step 3), plus the "Approval agent approves on its own" path (not every run should need a human).

Your files after Step 5 (final):

```
project_3_documind_rag_agent/
├─ state.py                → includes found chunks, not just the question
├─ graph.py                → full routing: search? → answer → approval → int
├─ retriever.py            → reused from 08_rag
├─ agents/
│   ├─ retriever_agent.py  → search + relevance-judging node
│   ├─ reasoner_agent.py   → grounded-answer node
│   └─ approval_agent.py   → automatic checks + interrupt() gate
└─ test_project3.py
```

Final Deliverable: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely

— a 3-agent LangGraph system. A Retriever judges its own search quality. A Reasoner answers only from grounded material. An Approval agent auto-clears safe drafts and pauses for a human on the rest.

Why this really is multi-agent (said plainly): three separate roles, each a real node (or sub-graph) with its own job and its own pass/fail judgment — not one big node doing everything. There's still no supervisor here (routing is a fixed conditional path, not a dynamic `Command`-based router) — that dynamic routing belongs to a more advanced Supervisor-based multi-agent system.

Checklist Before You Call This Done

- [] `StateGraph` with typed state, at least one real conditional edge
- [] Checkpointer connected — state survives a pause/resume cycle
- [] Retriever agent only runs when needed, not always, and judges its own search results

- [] Reasoner agent produces answers grounded in what the Retriever actually found
- [] Approval agent approves low-risk drafts on its own, and only asks a human via `interrupt()` when needed — both paths tested
- [] Full search → answer → approve/pause → resume → grounded answer cycle works start to finish
- [] You can draw the graph from the code (or the other way around), without running it

Full requirements, test cases, and hints: see the Steps above and the `hints_and_solutions/` files in this project.

Status

Not started. Track your own progress however works for you.